3C · AI CODING AT TEAM SCALE

How a skills library ships software.

Five diagrams of the workflow behind **3C** (Claude Code Coach): which skill dispatches which agent, the path every issue takes, the gates it passes, the context each step reads, and how work stays on one branch at a time.

01 → 05

Five diagrams. One workflow. Every agent and skill drawn here is one you can install.

Read-only agents

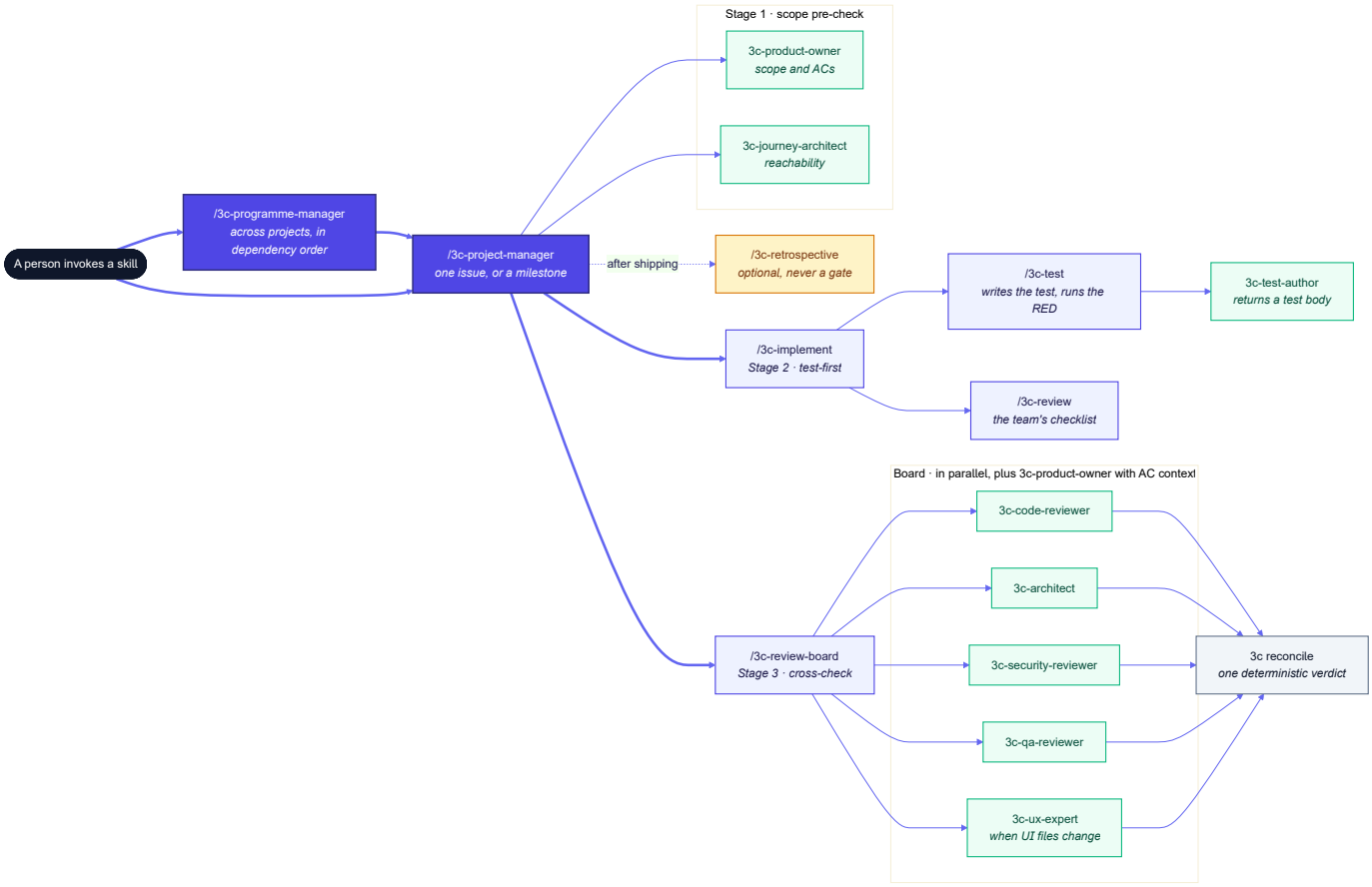
The agents read the change and report. Skills write, in your session, and a person confirms the branch and the PR.

MIT

Open source. The skills and agents install into `.claude/` and run in Claude Code.

Skills dispatch agents. Agents never dispatch.

Who calls whom. A person invokes a skill; the skills orchestrate in your session; the agents are dispatched one level deep, read the change, and report.

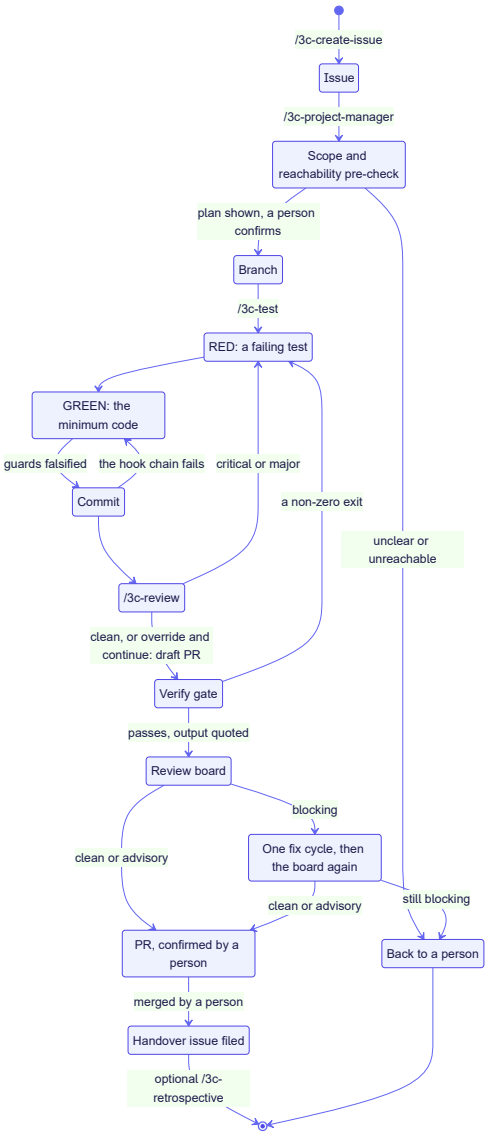


KEY MESSAGE

Orchestration lives in skills, not agents. **/3c-project-manager** checks scope first, implements test-first through **/3c-implement**, then convenes **/3c-review-board**. Its agents report in parallel to **3c reconcile**, so the verdict is computed, not written.

Every issue travels the same path.

One issue through `/3c-project-manager`, as a state machine: from the scope pre-check to the handover issue.

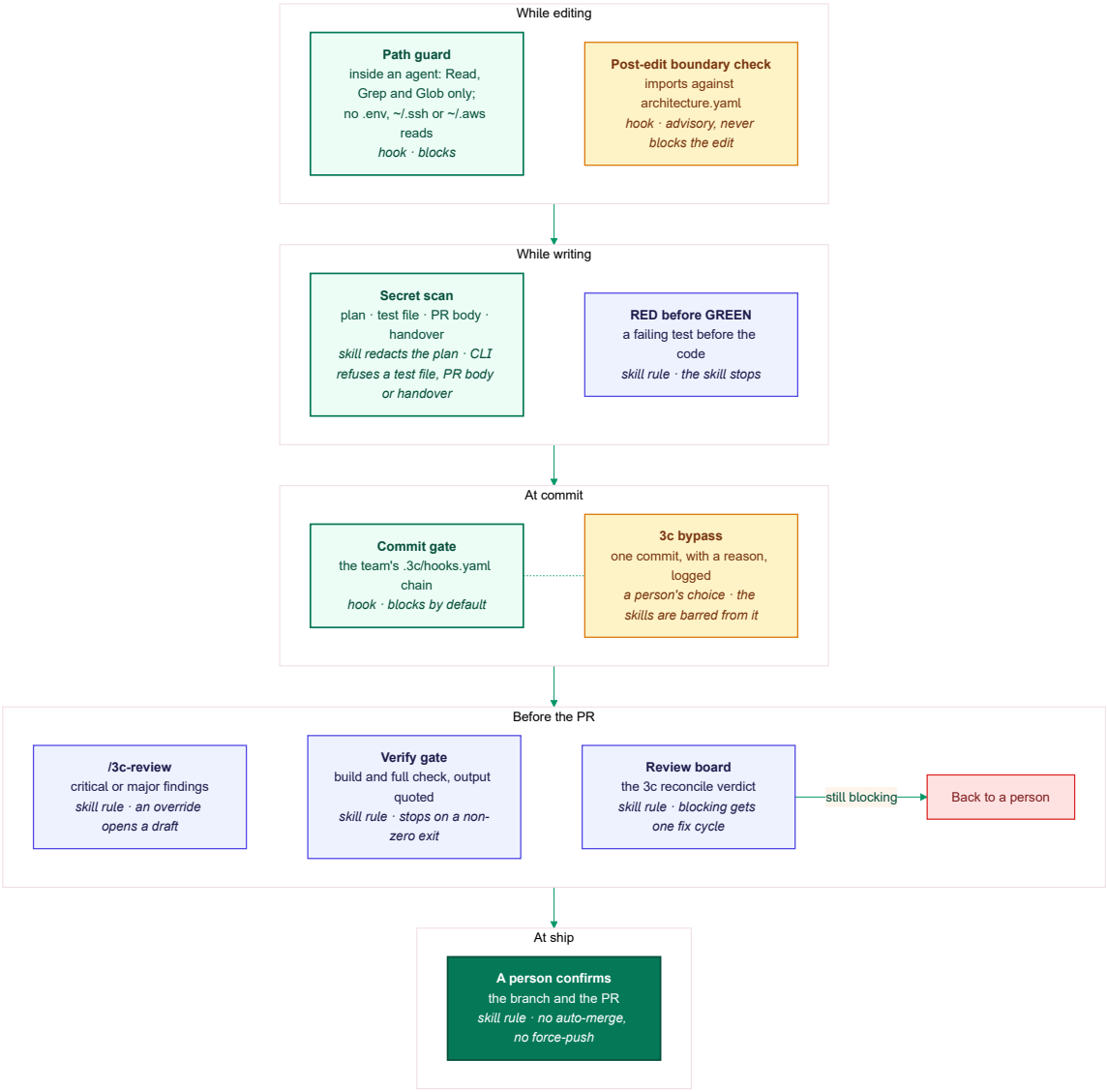


KEY MESSAGE

A person confirms the branch and the PR, and a person merges. Everything between is test-first and gated. A blocking verdict gets **one** fix cycle; if the board still blocks, the issue goes back to a person instead of round the loop again.

Gates over guidelines.

Each gate, grouped by when it fires, and marked for who enforces it: a hook, the CLI, or a rule the skill follows.

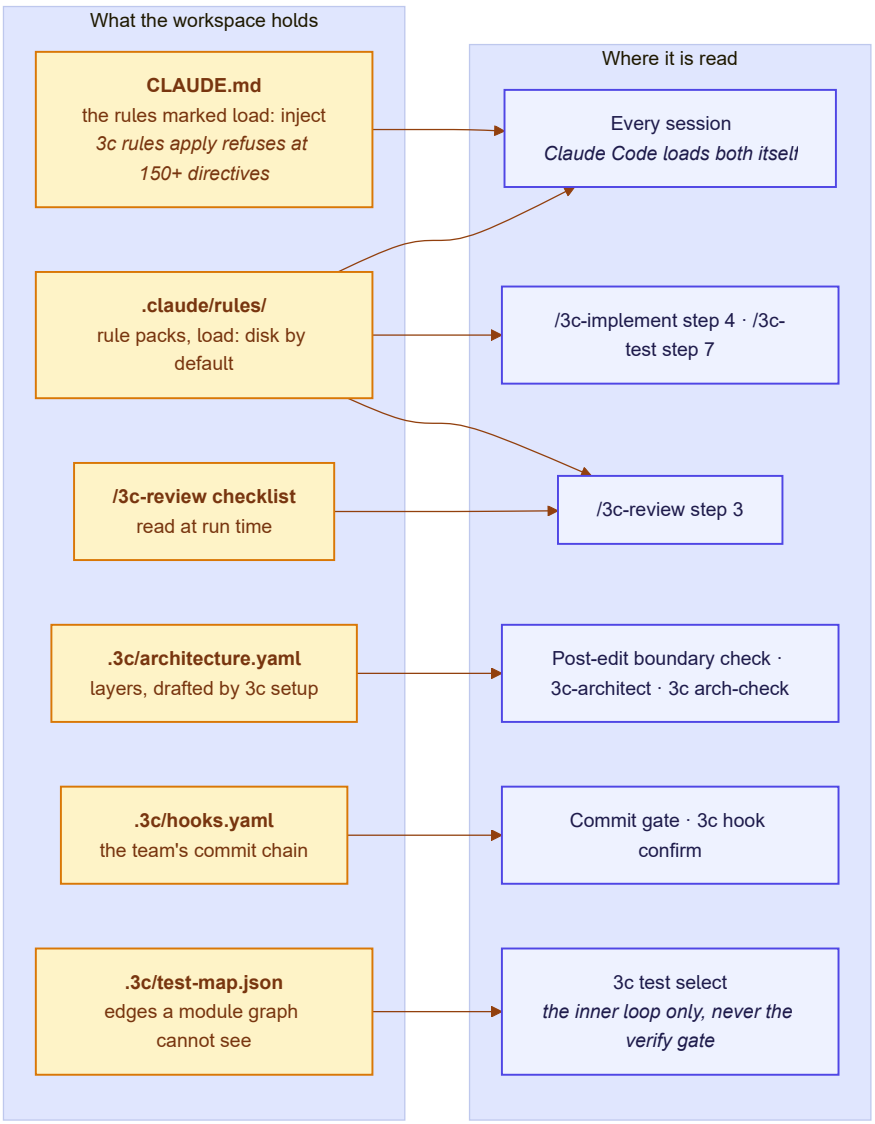


KEY MESSAGE

Every gate names who enforces it. Hooks block on their own: the path guard inside an agent, and the commit gate. The CLI refuses a test file, PR body or handover that carries a secret. The skills stop at their own rules: a failing test first, a clean review, a passing verify gate, a board that does not block. The post-edit boundary check only advises. **3c bypass** skips one commit, with a reason, and is logged.

Context, loaded where it matters.

What each context file holds, and the step that reads it.

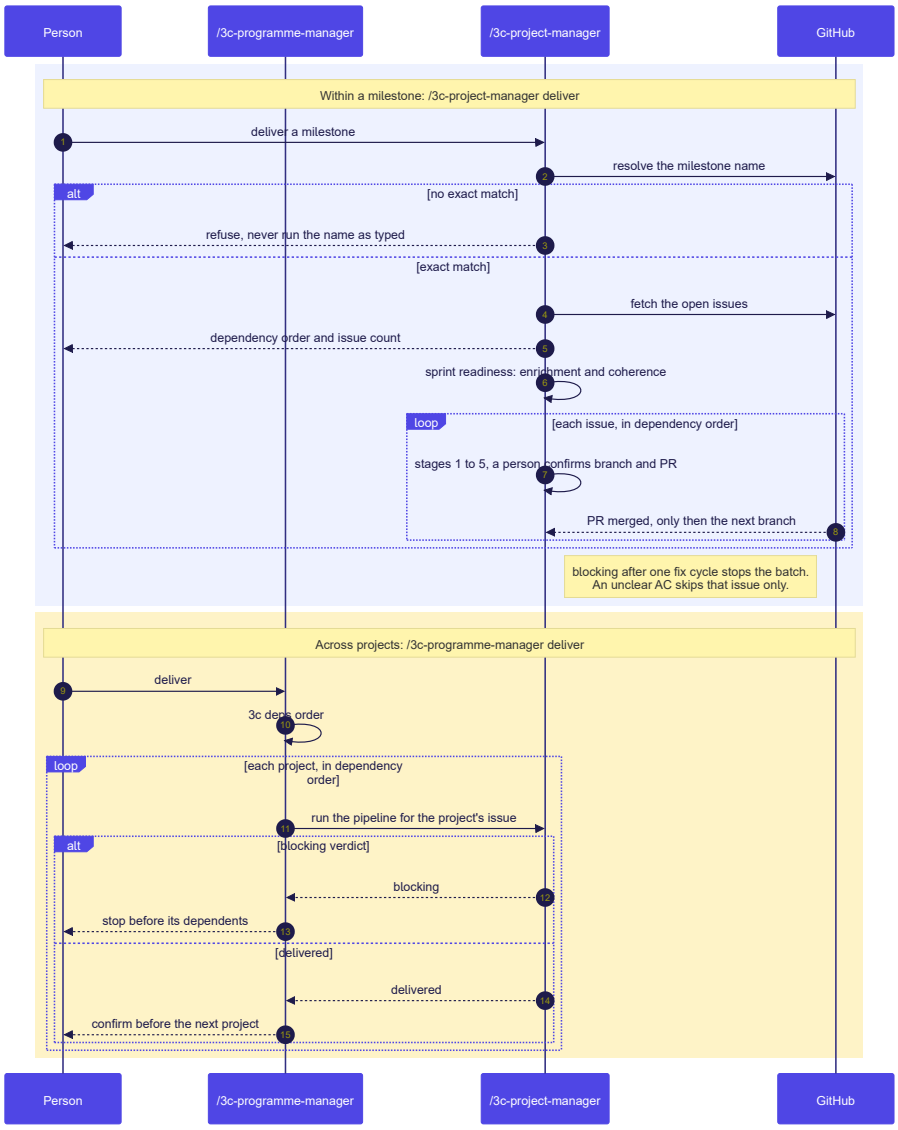


KEY MESSAGE

CLAUDE.md has a budget. Rule packs stay in `.claude/rules/` by default, where Claude Code loads them itself. Only rules marked `load: inject` are written into CLAUDE.md, and **3c rules apply** refuses at 150 or more directives. The checklist, the architecture map, the commit chain and the test map are each read by the step that needs them.

One branch at a time.

Serialisation at two levels: issues within a milestone, and projects across a workspace.



KEY MESSAGE

Parallel branches over a shared surface produce conflicting edits to the same files. So **/3c-project-manager deliver** opens issue N+1's branch only after issue N's PR has merged, and **/3c-programme-manager deliver** asks before each next project and stops before the dependents of a blocked one.

THE ONE-LINE SUMMARY

Governed AI coding is a workflow, not a hope.

Skills orchestrate. Agents read and report. Every gate names who enforces it. A person confirms the branch and the PR.

The retrospective is optional and never a gate. It records what worked, what didn't, and the action items to take next.

01
Name every role. Every agent and skill drawn here is one you can install.

02
Draw the lifecycle. If you can't draw it, you can't audit it.

03
Gates over guidelines. Every gate names who enforces it.

04
Give CLAUDE.md a budget. Read the rest where it is needed.

05
One branch at a time. The next branch waits for the last merge.

06
Bound the board. One fix cycle, then a person.

Open source, MIT licensed.
Borrow, fork, improve.

github.com/kgn-git/3C · skills and agents for Claude Code

#AIEngineering
#DeveloperExperience
#ClaudeCode
#SoftwareProcess
#Governance